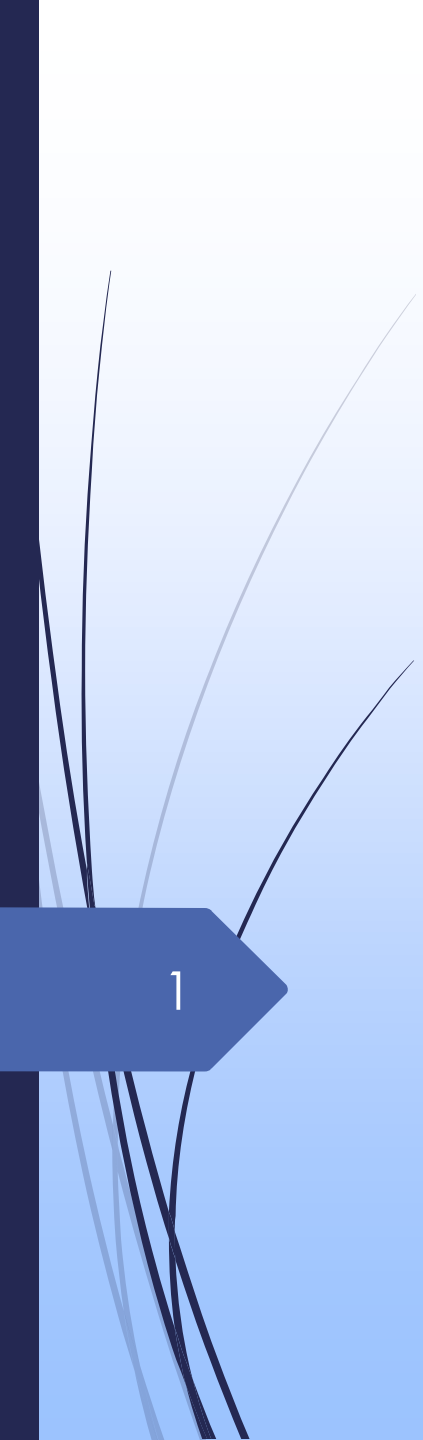




EECS 280 – Lecture 1

Course Introduction, Intro to C++

1

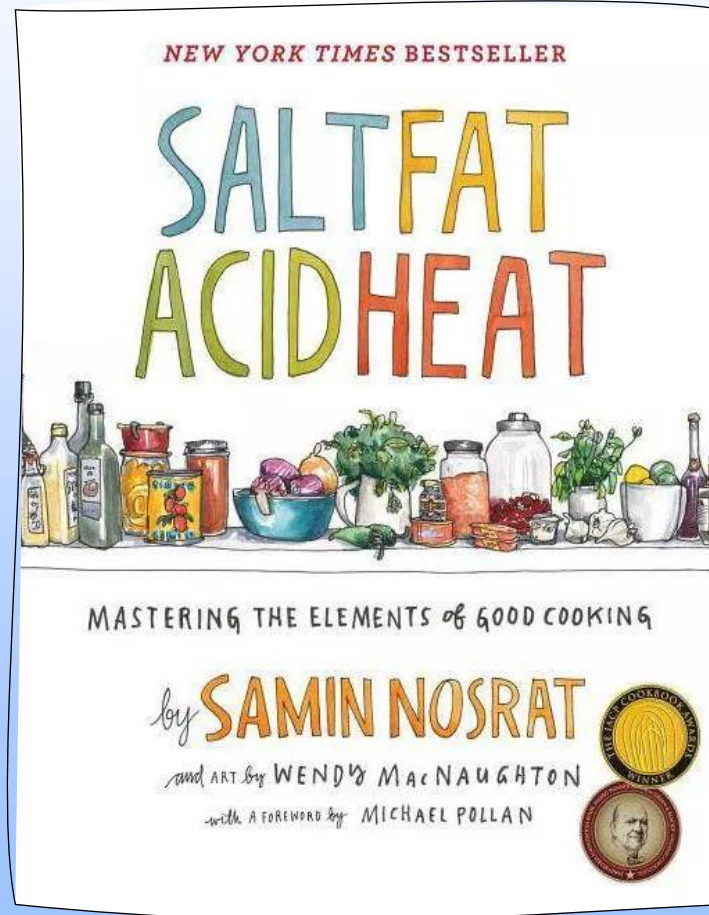
About Me – James Juett

- Ph.D. UMich 2016
- CSE Teaching Faculty
- Academic interests:
 - CS Education
 - Interactive Program Visualization
 - Principles of Software Design
- Other interests:
 - Spicy foods
 - Gardening
 - Cats
 - Escape games



What EECS 280 is about...

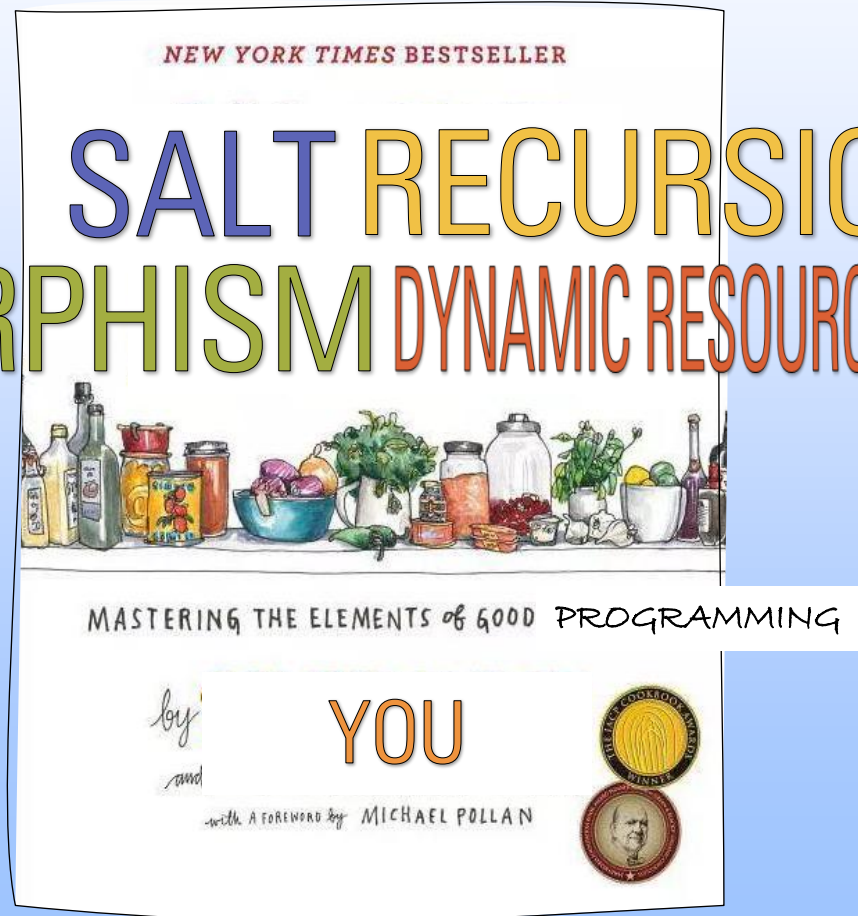
- ➔ **Generalizable** CS concepts



What EECS 280 is about...

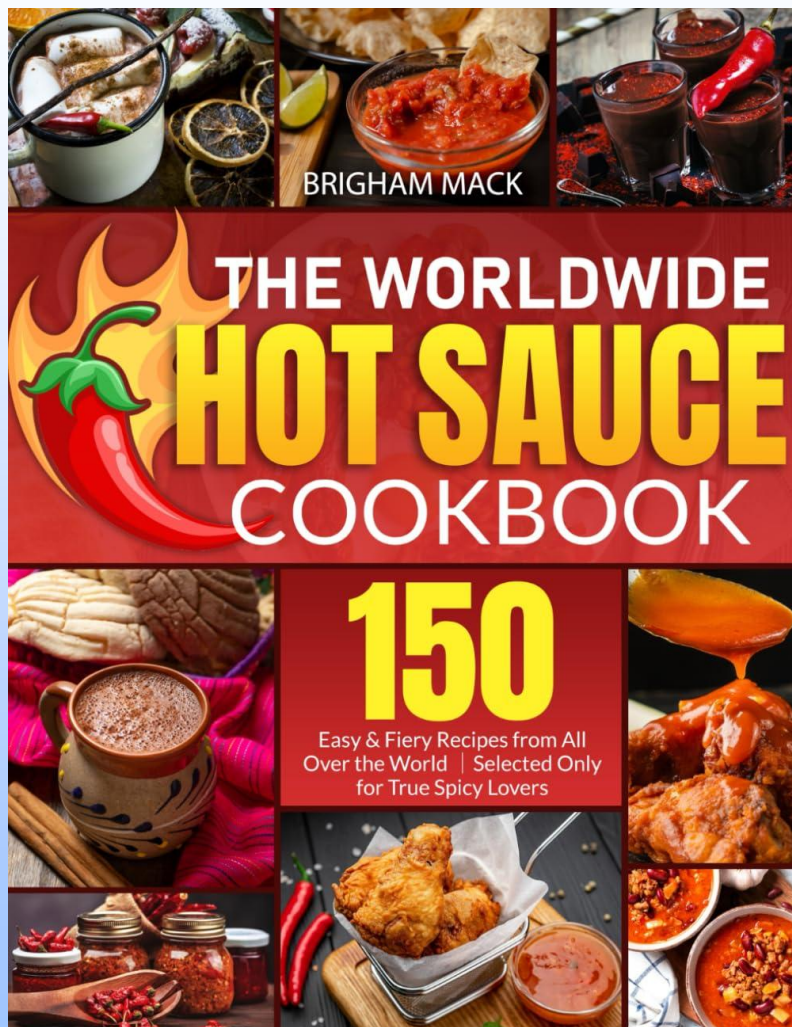
- ➔ **Generalizable** CS concepts

SALT RECURSION POLYMORPHISM DYNAMIC RESOURCE MANAGEMENT



Check out our syllabus for a list of lots of these concepts!

Another Book



What EECS 280 is about...

- **Generalizable** CS concepts
- Building programming skills

Projects – Learn By Doing

Project	Application	Concepts
1	Calculate Statistics	Procedural Abstraction
2	Image Processing	Memory and Pointers Data Abstraction
3	Game of Euchre	Classes, ADTS Inheritance and Polymorphism Object-Oriented Programming
4	Piazza Post Classifier	Containers and Iterators Open-ended Design
5	Text Editor	Linked Data Structures Dynamic Resource Management
6	Map Data Structure	Recursion Binary Search Trees

What's the goal?

- One of the best ways to learn how to program is to write programs.
- But the programs aren't the end goal.

You are!

What EECS 280 is about...

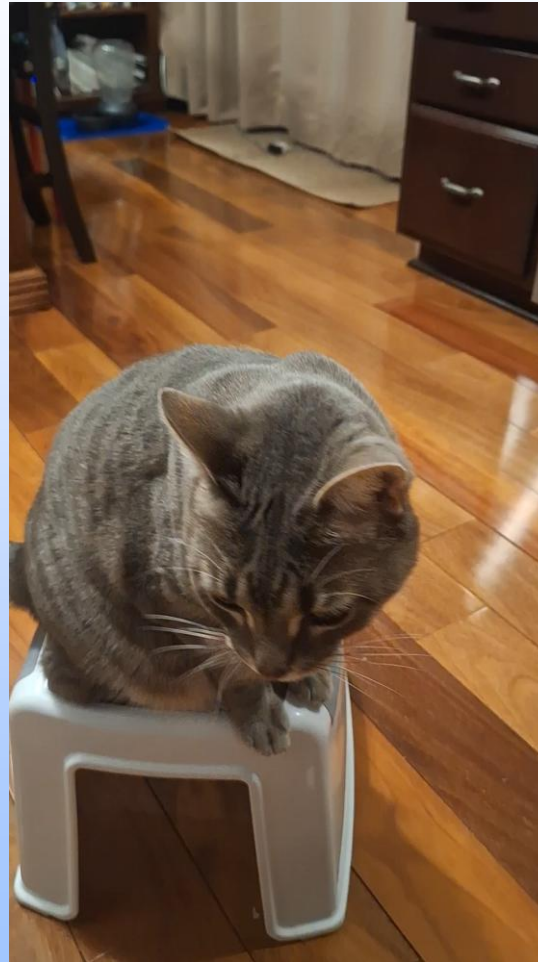
- Generalizable CS concepts
- Building programming skills
- Building our CS community

What's your AI-proof job?

➡ Cat trainer?

[LINK COMING SOON]

[QR Code]



eeecs280.org

OR

eeecs280staff.github.io/eeecs280.org

Resources

All linked from
eecs280.org

Course Notes

Textbook

Canvas

Announcements and Grades

Gradescope

Submit Labs, Review Graded Exams

autograder.io

Turn in Projects

eecs280.org

Sign up for office hours

Ed

Lecture, Lab, Project Q/A

Discord

Informal Course Community

Setup Tutorial

Get your computer set up for C++ development and EECS 280 projects.

1. Command Line Setup

 **Windows Setup**
Basic setup on Windows

 **macOS Setup**
Basic setup on macOS

2. Command Line Use

 **_Command Line**
Execute commands via shell

3. C++ Dev Environment

 **VS Code** Recommended
Easy-to use editor for any OS

 **XCode**
Feature-rich IDE for macOS

 **Visual Studio**
Feature-rich IDE for Windows

4. C++ Compilation

 **Makefiles**
Recipes for compiling and testing code

Step 1
Do this ASAP.
If you run into
issues, we
can help!

Steps 2 and 3
Consider attending
our C++ Setup
Walkthrough Session
before diving in.

Step 4
We'll also cover
Makefiles briefly
in lecture 2.

EECS 280 Setup Walkthrough

Join us for a live setup walkthrough session:

- ▶ Initial computer setup
- ▶ Using a command-line interface
- ▶ Using the VS Code Editor
- ▶ Workflow for compiling, running, testing, and debugging C++.

TBD, likely Tue/Wed evening via Zoom.

Separate, concurrent sessions for Windows/Mac.

Links on eecs280.org. Recordings will be posted.

For more details, watch the videos at
<https://eecs280.org/calculator.html>

15

Assignments

Lectures	3% optional	Labs	10%	Entry/Exit Surveys	1%	
Projects 44%	1	2	3	4	5	6
	4%	8%	8%	4%	8%	4%
Exams 50%	Midterm		Final			
	25%		25%			

- Project 3 includes an early checkpoint worth 0.5%.
- Score computed with and without lecture, take the better.

Grading Scale

1. Curve

- Only exams are curved
- Mean -> low 80s
- Std Dev -> smaller

2. Minimum Pass Threshold

- To earn a C or better, you need:
- $\geq 60\%$ project avg
- $\geq 65\%$ exam avg (after curve)

3. Final Grades

Determined by straight scale.
We do not round grades.
We do not expect to adjust thresholds.

Overall %	Grade
0 - 50%	E
50 - 60%	D
60 - 70%	C-
70 - 77%	C
77 - 80%	C+
80 - 83%	B-
83 - 87%	B
87 - 90%	B+
90 - 93%	A-
93 - 97%	A
97 - 100%	A+

Lecture Participation

- ▶ Earn lecture participation points by:
 - ▶ Attending and participating in live lecture
 - ▶ Completing embedded exercises in async lecture
- ▶ Grading
 - ▶ 1 point per lecture
 - ▶ Only need 20 points for full credit (can't get > 100%)

Details: <https://eecs280.org/syllabus.html#lecture-participation>



i'm looking for a meme to encourage students to keep up with lectures. any suggestions?



Here's an idea for a meme to encourage students to stay on top of lectures:

Visual: A picture of a student drowning in a sea of papers and textbooks, with text bubbles labeling them "Missed Lectures" and "Assignments Due."

Caption: "Skipping one lecture feels harmless... until you're buried alive in Week 12."

Let me know if you'd like more suggestions or help designing the meme!

uhh



Got it! Let's make it lighter and more fun:

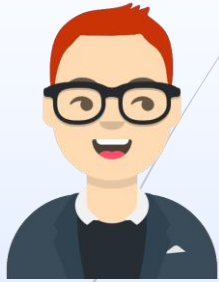
Visual: The "Distracted Boyfriend" meme.

- **Boyfriend (student):** "Skipping lectures."
- **Girlfriend (responsibility):** "Understanding the material."
- **Other girl (distraction):** "Procrastination."

Caption: "Don't let this be you! Stay loyal to your lectures."

How about this one?

Labs



You



Group A



Group B

- **Attendance is required for credit.**

- We'll assign you lab groups A and B for the term.

Lab	1	2	3	4	...
Group	A	B	A	B	...

- IAs and GSIs facilitate labs and are there to help!

Lab exercises via a **worksheet**.

Practice exercises, tutorials, and mini-projects.



IAs and GSIs

Labs are worth 5 points each.

- 1st point = arrive on time (5 min grace period)
- The worksheet contains several **checkpoints**.
When your group finishes each part, a staff member will check that your solutions are complete and satisfactory.
- Your group must also **submit** a completed worksheet PDF to **Gradescope**, due 8pm the calendar day after your lab.

Autograder

autograder.io

Turn in Projects

Implementation Tests

- 1. Public (released with project)*
- 2. Private (no feedback until after deadline)*

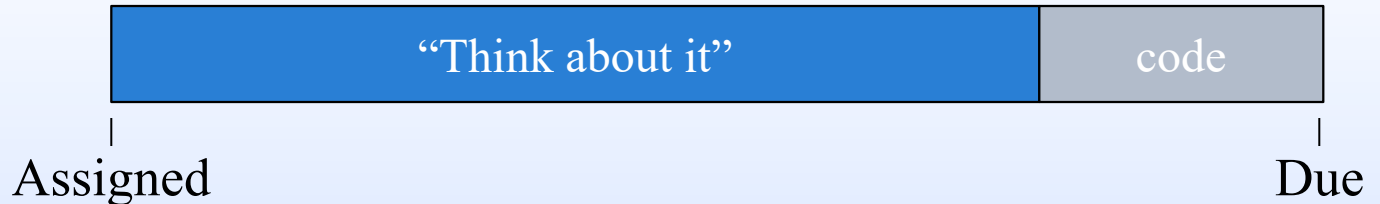
- 3. Mutation Tests (tests of your tests)*
See # of bugs caught. Don't need all to get 100%.

4. Style Checks

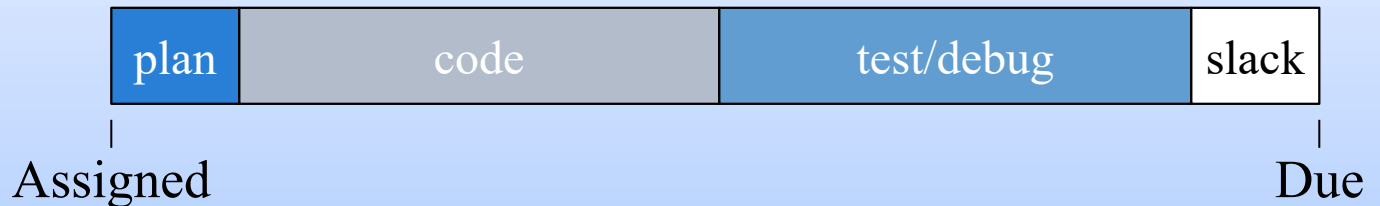
- The **public tests are not comprehensive**.
It's important to write your own tests!
- You get **3 submissions/day**. We grade your **best** one.
Submit **early** and submit **often**!

Project Timelines

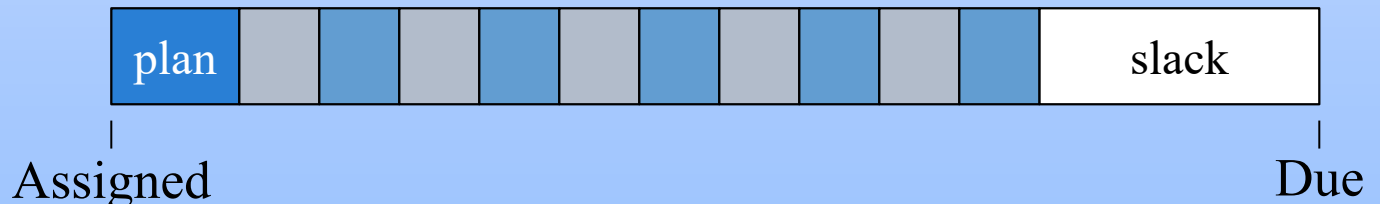
- Sample timeline for failure on projects.



- Sample timeline for success on projects.



- Even better!



Extensions

- ▶ We do not accept late work. Project and lab submissions are not allowed past the deadline.
- ▶ If you experience a medical or personal emergency, please reach out to us! We will consider exceptions on a case-by-case basis.
 - ▶ Please provide documentation if you are able – this helps us make appropriate accommodations.
- ▶ All requests for extensions must be submitted through the form linked from eecs280.org (not email).

We'll start again in five minutes.



Collaboration

Project 1	Individual
Projects 2-6	Partners Allowed
Assessments	Individual



	Lecture	Lab Teams	Ed	Study Groups
Everything Else	Collaboration Encouraged			

Project Partnerships

- Be **conscientious** about forming a partnership.
- Each student is **responsible** for the **complete** project.
- **Do not** split the work into pieces.
- Almost all your work should be done **together**.

Encouraged

Share **high-level** design strategies

Help others **understand** the project

Help others debug

Explain a compiler error to someone

Discuss testing **strategies**

Brainstorm edge case tests together

Using starter code or
example code given in lecture

Looking at someone else's code to
understand concepts

Prohibited

Share **step-by-step** code,
pseudocode, or **comments**

Share your solution as a “reference”

Debug someone's code **for them**

Fix a compiler error **for someone**

Share test case code
(even if not submitted)

Tell others what **specific tests**
they need to pass the autograder

Copy code in whole or in part, or
write code for someone else

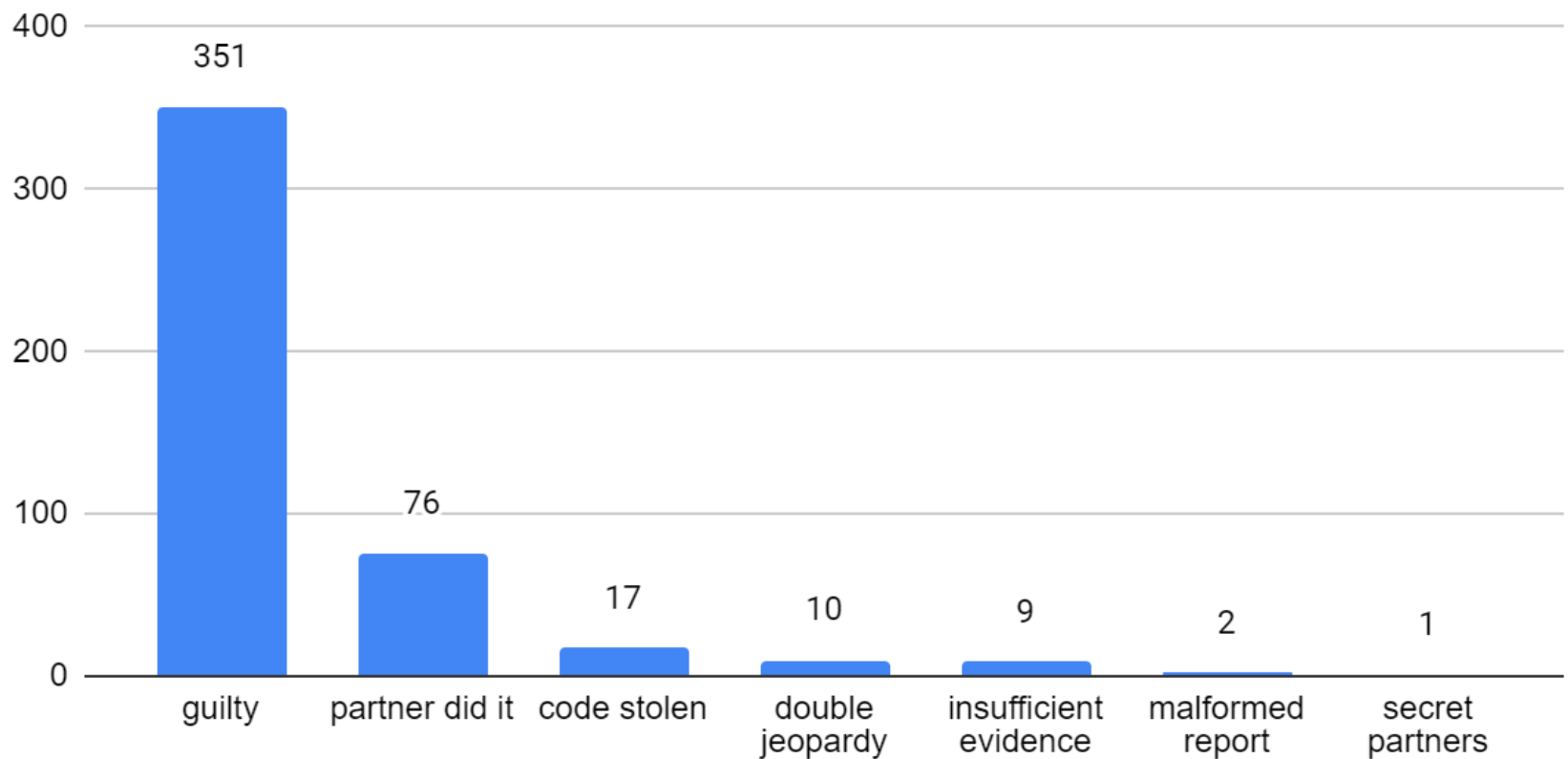
Share your code in a way
that could be **copied**

Cheat-Checking

- ▶ We use **automated software** and **manual inspection** to detect potential honor code violations.
- ▶ Over the past several years, we have **reported 2-10%** of the class each semester.
- ▶ We only report **clear cases** of academic dishonesty.
- ▶ You will **not** be reported for:
 - ▶ Using starter code provided by course instructors.
 - ▶ Having the same idea as someone else.
 - ▶ Receiving similar help/guidance from the same course staff member in office hours.

Cheat-Checking

Resolved EECS 280 HC Cases



Data from Sp18, W19, Sp20, W21, Sp21, F21, W22, Sp22, F22, W23

Generative AI Policy

- ▶ *The use of generative AI tools is permitted only in the same contexts where collaboration with another person is allowed.*

Allowed

Reinforcing course concepts, generating additional examples, interactively checking knowledge.

Explain compiler errors or get suggestions for debugging

Brainstorming and identifying potential edge cases for testing

Prohibited

Generating project code, test cases, or any other work to be submitted (with or without modifications)

Exhaustively proofreading your code to find and/or correct any mistakes

Asking for a list of test cases (even if they aren't written out as code)

Course Expectations

- **Respect others** and their opinions
- **Value people** and their diverse experiences
- Create an environment where **everyone** feels **safe and included**
- Take **learning opportunities** seriously
- **Stay connected**



31

What you'll get out of EECS 280

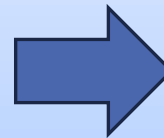
- Skills to design and write programs with 1,000+ lines of code
- Prerequisites for future computer science courses
- Credentials for an internship
- Become part of a community that is changing the world!

Machine Code

- ▶ Computers only run programs written in **machine code**.
- ▶ Each CPU uses a specific **Instruction Set Architecture (ISA)** that defines its **machine language**.
 - ▶ e.g. Intel Core i7 implements x86_64
 - ▶ e.g. Apple M1 implements AArch64

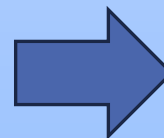
```
push    rbp
mov     rbp, rsp
mov     DWORD PTR [rbp-4], 3
mov     DWORD PTR [rbp-8], 4
mov     edx, DWORD PTR [rbp-4]
mov     eax, DWORD PTR [rbp-8]
add     eax, edx
...
```

X86_64



```
sub     sp, sp, #16
mov     w0, 3
str     w0, [sp, 12]
mov     w0, 4
str     w0, [sp, 8]
ldr     w1, [sp, 12]
ldr     w0, [sp, 8]
add     w0, w1, w0
...
```

AArch64

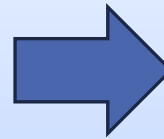


Compiled Languages

- Nobody wants to write machine code!
- Instead, write **source code** in a language like C++ and use a **compiler** to translate it to machine code.

```
int main() {  
    int x = 3;  
    int y = 4;  
    int z = x + y;  
}
```

C++ Source



```
push    rbp  
mov     rbp, rsp  
mov     DWORD PTR [rbp-4], 3  
mov     DWORD PTR [rbp-8], 4  
mov     edx, DWORD PTR [rbp-4]  
mov     eax, DWORD PTR [rbp-8]  
add     eax, edx  
...
```

X86_64

- A variety of compilers are available for C++
 - g++ (GNU C++ Compiler)*
 - Clang/LLVM
 - MSVC (Microsoft Visual C++)

*The EECS 280 autograder compiles with g++.

Brief Aside: Interpreted Languages

- ▶ Many languages are **interpreted** rather than compiled.
 - ▶ e.g. Python, Java, Javascript, etc.
- ▶ In this model, a host program called an **interpreter** is initially compiled to run on a particular architecture.
- ▶ Then, the interpreter reads your source code line-by-line, executing the appropriate machine code instructions.
- ▶ Example:
The EECS 280 grade calculator uses Javascript code to compute your grade. The code is actually being interpreted by your web browser, which is already compiled and running on your computer.

A First Program

- Let's take a look at a simple program in C++. It prints:

Hello World!
x=20 y=15

```
#include <iostream>
```

hello.cpp

```
int main() {  
    // print a greeting  
    std::cout << "Hello World!" << std::endl;  
  
    // Declare some variables  
    int x = 10 + 5;  
    int y = 0;  
    y = x;  
    x = 20;  
  
    // Print out the result  
    std::cout << "x=" << x << " y=" << y << std::endl;  
}
```

Example: Compile and Run C++ Code

1. Assume your source code is in a file `hello.cpp`
2. Run the compiler from the command-line using:

```
g++ --std=c++17 hello.cpp -o hello.exe
```

This creates an **executable** file called `hello.exe` that contains machine code compiled for your architecture.

3. Run the executable from the command line with:

```
./hello.exe
```

*For EECS 280 projects, you'll compile using the `make` utility and a custom `Makefile` we provide with the projects.

Demo

- ▶ Open an editor, write a hello world program
- ▶ Compile with g++ to hello.exe
- ▶ Run from terminal

Compile Errors

- If there are any **compile errors**, no hello.exe file is created.

```
#include <iostream>
```

hello.cpp

```
int main() {  
    // print a greeting  
    std::cout << "Hello World!" << std::endl;
```

```
    // Declare some variables
```

```
    int x = 10 + 5;
```

```
    int y = 0;
```

```
    y = x
```

```
    x = 20;
```

```
    // Print o
```

```
    std::cout
```

```
}
```

This is a syntax error.

```
jjuett@LAPTOP-N6606F3M:~/temp$ g++ hello.cpp -o hello.exe
```

```
hello.cpp: In function 'int main()':
```

```
hello.cpp:6:33: error: expected ';' before 'int'
```

```
6 |     cout << "Hello World!" << endl
```

```
^  
;
```

```
.....  
9 |     int x = 10 + 5;
```

```
    ~~~
```

```
hello.cpp:11:7: error: 'x' was not declared in this scope
```

```
11 |     y = x;  
    ^
```

Statements

- ▶ C++ is an **imperative** programming language – we provide a sequence of **statements** to run in order.
- ▶ A program starts execution in the `main` function.

```
#include <iostream>
```

```
int main() {
```

```
    // print a greeting
```

```
    std::cout << "Hello World!" << std::endl;
```

```
    // Declare some variables
```

```
    int x = 10 + 5;
```

```
    int y = x;
```

```
    y = x;
```

```
    x = 20;
```

```
    // Print out the result
```

```
    std::cout << "x=" << x << " y=" << y << std::endl;
```

```
}
```

Statements in
C++ end with a
semicolon.

Printing Output

- To print output to the terminal, send it to `cout` with `<<`.
 - Pronounced “see out”, not “cowt” 😊.
 - `endl` prints a newline and “flushes” the output.

```
#include <iostream>
```

Use the **#include** `<>` directive to import functionality from **header files** in the **C++ Standard Library**.

```
int main() {
```

```
    // print a greeting
```

```
    std::cout << "Hello World!" << std::endl;
```

```
    // Declare some variables
```

```
    int x = 10 + 5;
```

```
    int y = x;
```

```
    y = x;
```

```
    x = 20;
```

```
    // Print out the result
```

```
    std::cout << "x=" << x << " y=" << y << std::endl;
```

```
}
```

Namespaces

- Names in C++ may be organized into **namespaces**.
- Names from the C++ Standard Library are in the `std` namespace.
- Use **qualified names**:
`std::cout`
`std::endl`
- Or the **using** keyword to allow unqualified names.

```
#include <iostream>
int main() {
    // print a greeting
    std::cout << "Hi!"
               << std::endl;
}
```

```
#include <iostream>
using std::cout;
using std::endl;

int main() {
    // print a greeting
    cout << "Hi!" << endl;
}
```

```
#include <iostream>
using namespace std;

int main() {
    // print a greeting
    cout << "Hi!" << endl;
}
```

Formatting

- ▶ Spaces and line breaks don't affect functionality.
- ▶ They DO make your code more readable.
 - ▶ *Tip: Many editors have auto-formatters built-in.*

```
#include <iostream>
using namespace std;

int main() {
    // print a greeting
    cout << "Hello World!" << endl;

    // Declare some variables
    int x = 10 + 5; int y = 0; y = x; x = 20;

    // Print out the result
    cout << "x=" << x << " y=" << y << endl;
}
```

Several
statements
on one line

Inconsistent
indentation

Comments

- ▶ `//` The rest of the line is a comment
- ▶ `/*` and `*/` Any code between is a comment
- ▶ IDEs have keyboard shortcuts for comment/uncomment

```
#include <iostream>
using namespace std;

int main() {
    // print a greeting
    cout << "Hello World!" << endl; // another comment

    // Declare some variables
    int x = 10 + 5;
    int y = x;
    /*y = x;
    x = 20;*/

    // Print out the result
    cout << "x=" << x << " y=" << y << endl;
}
```

Expressions

- Expressions generally have one of two purposes.*

1. To compute something: `10 + x + sqrt(x);`

- We say an expression **evaluates** to the compute value.

2. To do something: `cout << "hello";`

`x = 10;`

- We say the expression has a **side effect**.

- Expressions may be composed of:

Literals

`10`

Variables

`x cout`

Function Calls

`sqrt(x);`

Operators

`+ - >= << % !
++ [] -> new`

*Some expressions have *both* a significant side effect and evaluated result.

Variable Declarations

- In C++, you must explicitly **declare** variables with their:
 - 1. Type 2. Name 3. Initializer expression (optional)

```
// Declare some variables
int x = 10 + 5;
int y = 0;

// OK - can use x and y now
y = x;
x = 20;
cout << y;

// COMPILE ERROR - what is z???
z = 20;

// COMPILE ERROR - can't declare x again
int x = 20;
```

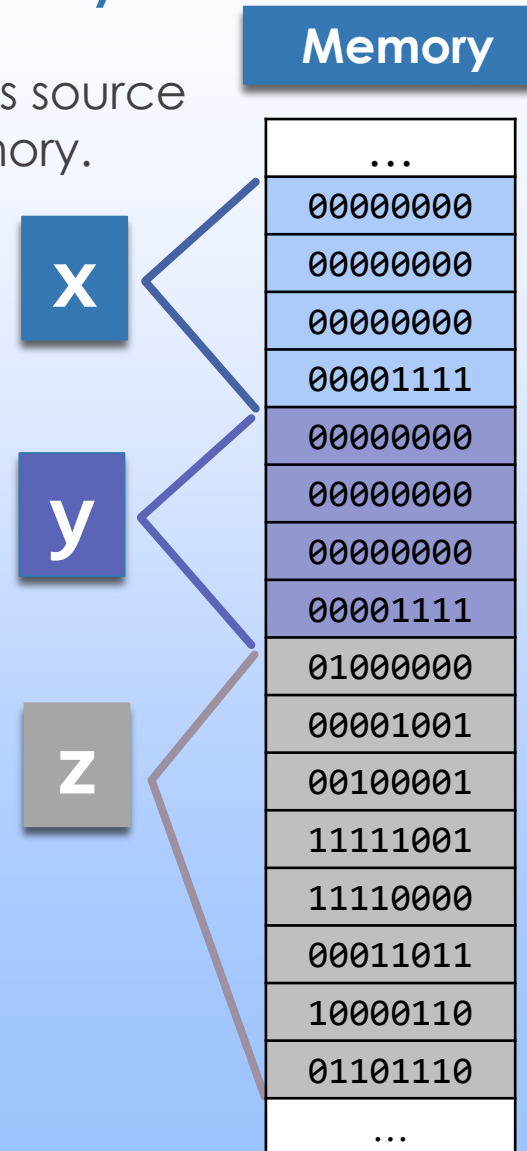
Write the type when you declare it, but NOT when you use it.

Variables and Memory

- A variable is a **name** in the program's source code that refers to an **object** in memory.

```
// Declare some variables
int x = 10 + 5;
int y = x;
double z = 3.14159;
```

- Data is stored in **bytes**, composed of 8 **bits** - binary digits of 0 or 1.
- A variable's **type** determines:
 - How much space it needs
e.g. `int` = 4 bytes, `double` = 8 bytes
 - How its value is encoded to binary



Uninitialized Variables

- **Warning!** Initialize variables before using them.
 - Unfortunately, the compiler can't always check this for you.

```
cout << x; // COMPILE ERROR

// x is uninitialized - has some "random" value
int x;

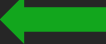
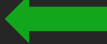
// Runtime Error - Compiles successfully, but
// has undefined behavior when the program runs.
cout << x;

x = 3; // OK - x now has value 3
cout << x; // Behavior is now well-defined
```

- Some errors can be caught at **compile time** whereas others don't necessarily occur until **runtime**.

Scope

- A **scope** is a region of the source code where **names** are meaningful.
- Often defined by a **block** using { }
- A variable may be used only if it is **in scope** - from its declaration to the end of the *nearest* enclosing scope.
- Variables in **different scopes** with the **same name** are unrelated.

```
int add(int x, int y) {  
    int result = x + y;  
    return result;  
}  
  
int main() {  
    int x = 5;  
  
    cout << x;   
  
    int y = add(x, 4);  
    cout << y;   
    cout << result;   
  
    if (y > 5) {  
        int a = y + 100;  
        cout << x;   
        cout << a;   
    }  
  
    cout << a;   
}
```

Minute Exercise – which of these cause a compile error?

Static Typing in C++

- All variables and expressions have a fixed **static type**.

```
// Variables
```

```
int x = 10; // an int forever and always
```

```
string str = "llama"; // a string forever and always
```

```
... // Code may change the value of x or str, but it's
```

```
... // impossible to change their types.
```

```
// Expressions
```

```
x * 2; // value = ??? but always an int
```

```
x < 5; // value = ??? but always a bool (i.e. true/false)
```

```
str + str; // value = ??? but always a string
```

- The compiler's **type system** detects type errors.

```
str * 2; // always INVALID OPERATION
```

```
x = "cat"; // always INVALID CONVERSION
```

C++ Fundamental Types

- Fundamental types are built-in to the C++ language and always available.

Type	Description	Example
int	Signed integer	<code>int x = 3;</code>
double	Floating-point number*	<code>double y = 2.5;</code>
bool	“Boolean” values, i.e. true/false	<code>bool z = true;</code>
char	Single character	<code>char c = 'w';</code>
...	...	...

*There is also a floating-point type called `float`. A `double` is so-named because it generally uses twice the memory space of a `float` to allow greater precision.

Reprise: Static Typing in C++

- All variables and expressions have a fixed **static type**.

```
// Variables
int x = 10; // an int forever and always
double y = 3.14; // a double forever and always
string str = "llama"; // a string forever and always
```

- The compiler's **type system** can catch type errors.

```
str * 2; // always INVALID OPERATION
x = "cat"; // always INVALID CONVERSION
```

- 
- In other cases, an **implicit conversion** may be performed.

```
x = y; // double -> int conversion. 3.14 -> 3
```

Implicit Conversions

- In some cases, the compiler allows an expression to be **implicitly converted** to a different type.

- `int -> double` `3` → `3.0`
Value unchanged. (A widening conversion.)

- `double -> int` `3.7` → `3` `-3.7` → `-3`
Value is *truncated*. (A narrowing conversion.)

- `numeric -> Boolean`
Zero is false. Anything else is true.

`4.2` → `true`

`-1` → `true`

`0` → `false`

Exercise: Implicit Conversions

- ▶ The program below contains many implicit conversions. Find them. Some cause bugs – which ones, and why?

```
double max(double x, double y) {  
    int result = x;  
    if (y > x) {  
        result = y;  
    }  
    return result;  
}  
  
int main() {  
    double x = 5;  
    double y = 7.4;  
    double z = max(x, y);  
    cout << z << endl;  
}
```

Explicit Conversions (Casts)

- In some cases, it may be necessary or more stylistically appropriate to use an explicit cast.

```
int main() {  
  
    double value = 4.3;  
  
    // implicit conversion, too easy to miss  
    int x = value;  
  
    // C-style cast, avoid doing this  
    int x = (int)value;  
  
    // C++-style cast, compiler checks  
    int x = static_cast<int>(value);  
  
}
```